

Author	Reviewer	Approver
BaoHT5		

Workshop 3: Building a Chatbot System Using ChromaDB, OpenAI SDK, and HuggingFace Text-to-Speech

Workshop Objective

Guide participants through the process of building a complete chatbot system that:

- Stores and retrieves knowledge using **ChromaDB** (a vector database for storing and searching embeddings).
- Uses the **OpenAI SDK** to generate natural language responses.
- Converts text responses into speech using a **HuggingFace Text-to-Speech model** (e.g., VITS model) to allow audio responses.

By the end of the workshop, teams will create a functional chatbot that answers queries, stores information in ChromaDB, and responds audibly using TTS (Text-to-Speech) technology.

Open-ended Question

As a team, build a chatbot system that integrates:

- **ChromaDB** for persistent, embedding-based storage of information (e.g., FAQs, knowledge base).
- **OpenAI SDK** to process user inputs and generate natural language responses.
- **HuggingFace Text-to-Speech** to convert the chatbot's responses into audio, providing a richer user experience.

Challenge: The chatbot should be able to interact with users, retrieve stored knowledge from ChromaDB, and speak the responses aloud.

Applications (Team Choice Examples)

- **Customer Support Chatbot:** Automatically responds to customer queries and speaks the response.
- **Educational Tutor Bot:** Answers student queries and provides audible explanations.
- **Entertainment Recommendation Bot:** Suggests TV shows or movies (could use Netflix data for recommendation).
- **Personal Finance Assistant:** Helps users track their expenses, answer finance-related queries, and speak out financial summaries.
- **Language Learning Assistant:** Answer questions help with learning and speaks the responses in the target language.

AI Tools & Technologies Usage Guidelines

Tool/Tech	Usage
ChromaDB	Persistent storage of knowledge using vector embeddings
OpenAI SDK	Language model for conversational responses and query processing
HuggingFace TTS (VITS model)	Converts text responses into speech using a pretrained model

Teamwork Approach

- Teams of 4-5 participants will collaboratively develop all parts of the chatbot system.
- There are **no predefined roles**; every member will contribute to all aspects including ideation, coding, and testing.
- The focus is on combining **data storage**, **language generation**, and **speech synthesis** into a working application.
- Teams can develop in a **shared Jupyter Notebook** or a **minimal web-based UI** to interact with the chatbot.

Agenda

1. **Define Chatbot Use Case & Generate Mock Data**
Choose the chatbot's purpose and generate mock data for storing in ChromaDB (e.g., knowledge base, FAQs, or user data).
2. **Set Up ChromaDB for Persistent Storage**
Use ChromaDB to create a collection, store mock data, and retrieve relevant information based on user queries.
3. **Integrate OpenAI SDK for Conversational Responses**
Use Azure OpenAI's chat model to process user input and generate dynamic, human-like responses.
4. **Integrate HuggingFace Text-to-Speech (TTS) for Voice Output**
Convert text responses from Azure OpenAI into speech using the VITS model from HuggingFace for real-time audio output.
5. **Test, Debug, and Optimize**
Test the full system for conversational accuracy, knowledge retrieval, and audio synthesis. Optimize responses and handle errors gracefully.
6. **Demo & Reflection**
Present the working chatbot system and share insights on challenges faced and how they were solved.

Deliverables by End of Workshop

- A fully functional chatbot with:
 - **ChromaDB** stores mock data (e.g., user queries, FAQs, knowledge base).
 - **OpenAI SDK** generates natural language responses.
 - **HuggingFace TTS** converting responses into spoken audio.
- A **working codebase** with a simple interface (CLI or web).
- **Tested conversation logs** demonstrating multi-turn interaction.
- A **demo** presentation showing the chatbot's capabilities and insights from the project.

Example Project: Entertainment Recommendation Bot

Project Brief

Build a chatbot that helps users find TV shows or movies based on their preferences. The bot should store popular titles and descriptions in ChromaDB and speak the recommendations aloud.

Code Example

```
# Step 1: ChromaDB setup and storing mock data
import chromadb
from openai import AzureOpenAI
import os

# Create a persistent client
client_chroma = chromadb.PersistentClient()

# Initialize AzureOpenAI client
client = AzureOpenAI(
    api_version="2024-07-01-preview",
    azure_endpoint=os.getenv("AZURE_OPENAI_ENDPOINT"),
    api_key=os.getenv("AZURE_OPENAI_API_KEY"),
)

# Create a collection using OpenAI Embedding function
collection = client_chroma.create_collection(
    name="netflix_titles",
    embedding_function=OpenAIEmbeddingFunction(model_name="text-embedding-3-small", api_key="<Azure_OPENAI_API_TOKEN>",
    api_base="<Azure_OPENAI_ENDPOINT>",
)

# Mock data (e.g., movie titles and descriptions)
netflix_titles = [
    {"title": "Stranger Things", "description": "A group of kids in the 80s uncover a government conspiracy."},
    {"title": "The Witcher", "description": "Geralt of Rivia, a monster hunter, fights dark creatures in a medieval world."},
    {"title": "Breaking Bad", "description": "A high school teacher turns to making methamphetamine to secure his family's future."},
    {"title": "Money Heist", "description": "A group of robbers carry out a meticulously planned heist on the Royal Mint of Spain."},
]

# Inserting mock data into ChromaDB collection
for movie in netflix_titles:
    collection.add([movie["description"]], metadata=[{"title": movie["title"]}])

def get_movie_recommendation(query):
    # OpenAI call to generate a movie recommendation based on user input
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": query}],
```

```

    )
    return response.choices[0].message["content"]

# Step 3: Integrating HuggingFace TTS for voice output
from transformers import VitsModel, AutoTokenizer
import torch

# Load the TTS model
model = VitsModel.from_pretrained("facebook/mms-tts-vie")
tokenizer = AutoTokenizer.from_pretrained("facebook/mms-tts-vie")

def text_to_speech(text):
    # Convert text to speech
    inputs = tokenizer(text, return_tensors="pt")
    with torch.no_grad():
        output = model(**inputs).waveform
    return output # This is the audio output

# Step 4: Putting it all together in a simple CLI-based chatbot

def run_chatbot():
    print("Welcome to the Entertainment Recommendation Bot! Ask for TV
shows or movies.")

    while True:
        query = input("You: ")
        if query.lower() in ['exit', 'quit']:
            break

        # Retrieve recommendation from OpenAI (based on user input)
        recommendation = get_movie_recommendation(query)
        print(f"Bot: {recommendation}")

        # Convert response to speech
        speech_output = text_to_speech(recommendation)
        print(f"Bot (audio): Playing recommendation audio...")

if __name__ == "__main__":
    run_chatbot()

```

Suggested Testing Scenarios

Test Case ID	Description
TC_01	User asks for a movie recommendation → bot responds with a relevant suggestion
TC_02	User asks for multiple recommendations → bot speaks out each suggestion
TC_03	Handle ambiguous queries → bot asks for clarification
TC_04	Test voice output with different responses and languages

References

- **ChromaDB** & **OpenAI** **Embeddings**
<https://www.chromadb.com/docs/>
<https://platform.openai.com/docs/guides/embeddings>
- **HuggingFace** **TTS** **Model** **(VITS)**
<https://huggingface.co/facebook/mms-tts-vie>
<https://huggingface.co/docs/transformers/>
- **OpenAI** **SDK**
<https://platform.openai.com/docs/guides/gpt>